3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

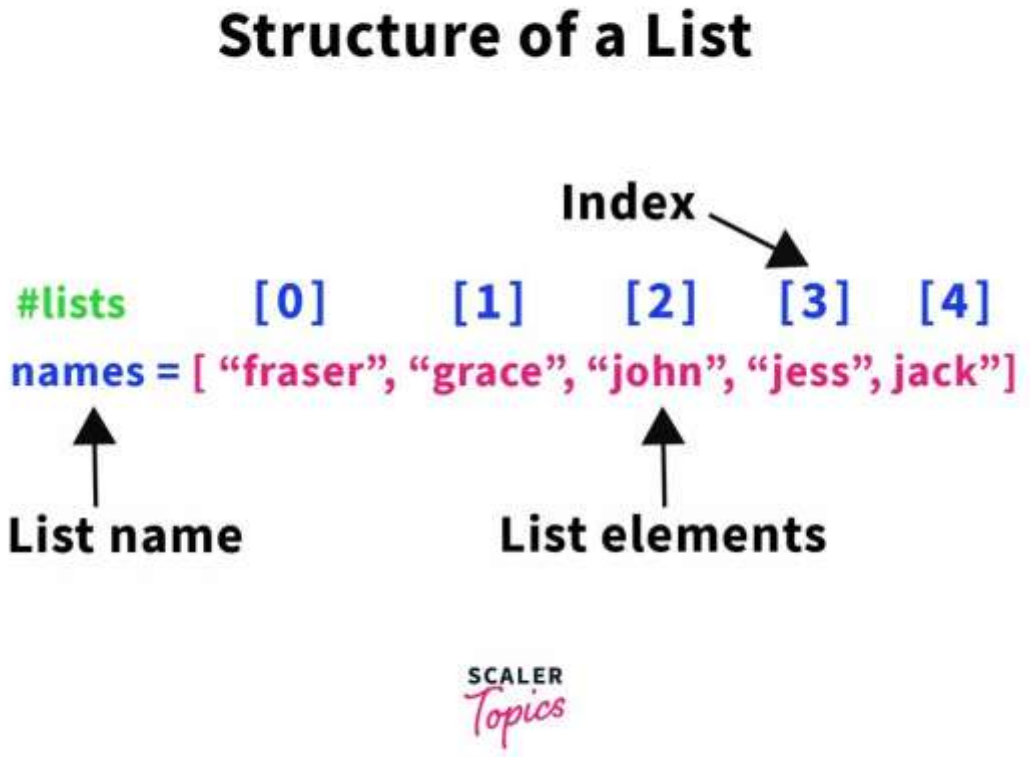
Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 5 – Lists & Advanced List Operations

FOUNDATION



Learning Objectives

By the end of this class, students will be able to:

- Understand what a Python **List** is.
- Create and access list elements.
- Use positive and negative indexing.
- Perform list slicing.
- Add, update, and remove elements.
- Use append(), extend(), insert(), remove(), and pop().
- Search and count elements in a list.
- Sort and reverse lists.
- Use sum(), max(), min(), and len().
- Calculate student total, average, highest and lowest marks.
- Build a **Student Marks Analyzer**.

3DWEBSOFT FOUNDATION

1. What is a List?

A **list** is a Python data structure used to store multiple values in a single variable.

Instead of creating separate variables:

mark1 = 85

mark2 = 92

mark3 = 78

mark4 = 90

we can store them together:

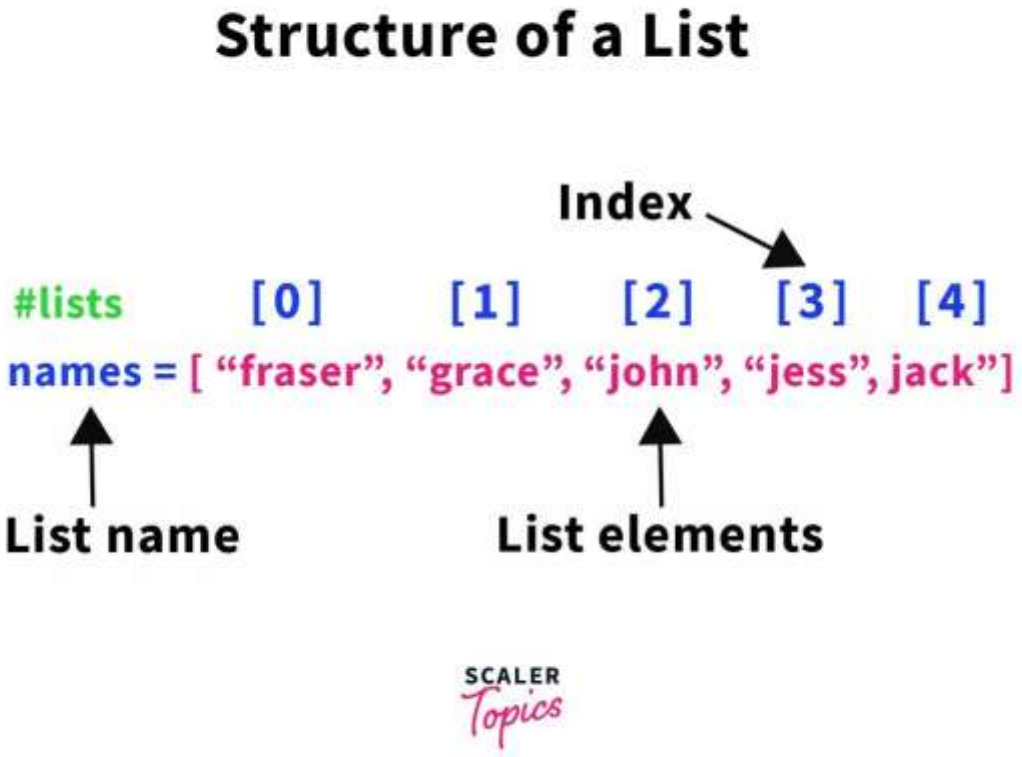
marks = [85, 92, 78, 90]

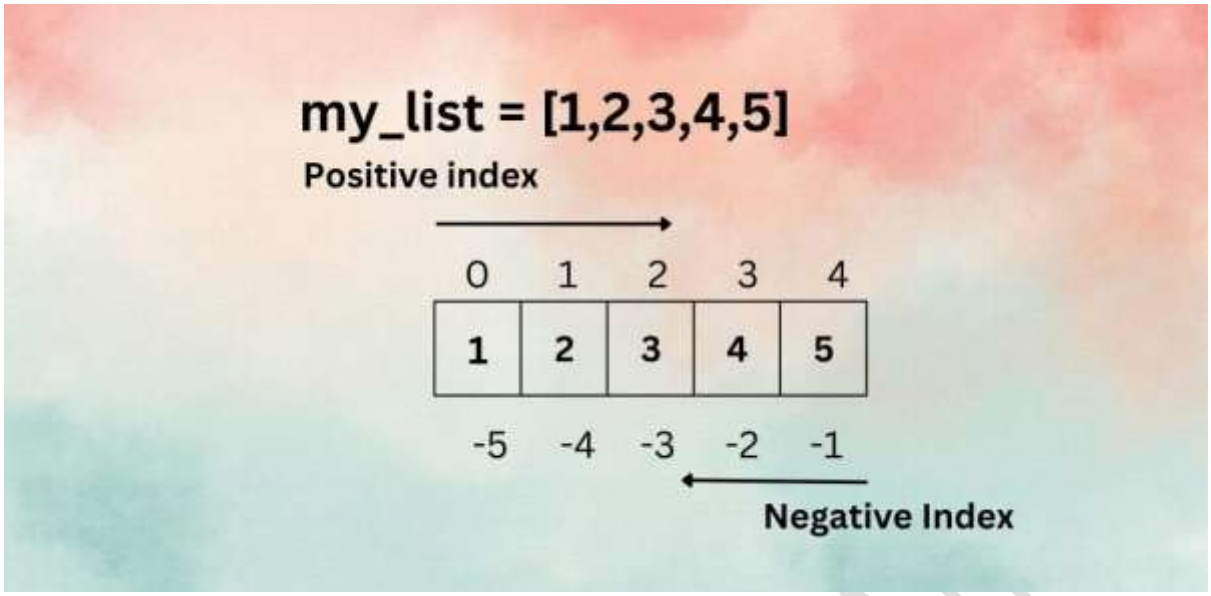
A list can contain:

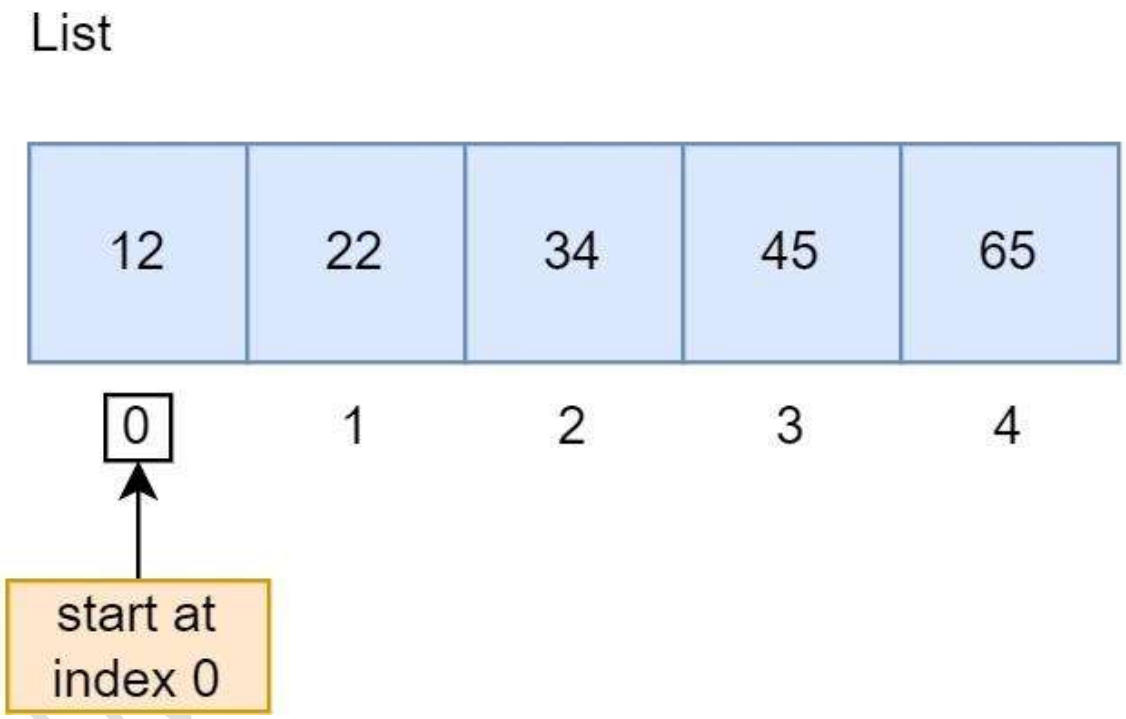
- Numbers
- Strings
- Boolean values
- Other lists
- Mixed data types

Example:

student = ["Aisha", 17, 92.5, True]







2. Important Characteristics of Lists

Python lists are:

1. Ordered

Elements maintain their position.

marks = [80, 90, 70]

The order is:

80 → 90 → 70

2. Mutable

Mutable means we can change the list after creating it.

marks = [80, 90, 70]

marks[1] = 95

print(marks)

Output:

[80, 95, 70]

3. Allow Duplicate Values

marks = [80, 90, 80, 70]

The value 80 can appear more than once.

4. Can Store Different Data Types

data = ["Rahul", 17, 85.5, True]

5. Dynamic Size

We can add or remove elements whenever required.

3. Creating Lists

Empty List

students = []

List of Numbers

3DWEBSOFT FOUNDATION

marks = [85, 90, 78, 92, 88]

List of Names

students = ["Aisha", "Rahul", "Priya", "Arjun"]

Mixed List

student = ["Aisha", 17, 92.5]

4. Accessing List Elements

Every list element has an **index number**.

Python indexing starts from **0**.

marks = [85, 90, 78, 92, 88]

Index	Value
0	85
1	90
2	78
3	92
4	88

Example:

```
print(marks[0])
print(marks[3])
```

Output:

85

92

3DWEBSOFT FOUNDATION

5. Negative Indexing

Python also allows indexing from the end.

Positive Index:

```
 0   1   2   3   4
 ↓   ↓   ↓   ↓   ↓
[85,  90,  78,  92,  88]
```

Negative Index:

```
-5  -4  -3  -2  -1
 ↓   ↓   ↓   ↓   ↓
[85,  90,  78,  92,  88]
```

Example:

```
marks = [85, 90, 78, 92, 88]

print(marks[-1])
```

Output:

88

6. List Slicing

Slicing is used to extract a portion of a list.

3DWEBSOFT FOUNDATION

Syntax:

list[start:end]

Example:

marks = [85, 90, 78, 92, 88]

print(marks[1:4])

Output:

[90, 78, 92]

Remember:

Start index is included, end index is excluded.

More Examples

marks[:3]

Output:

[85, 90, 78]

marks[2:]

Output:

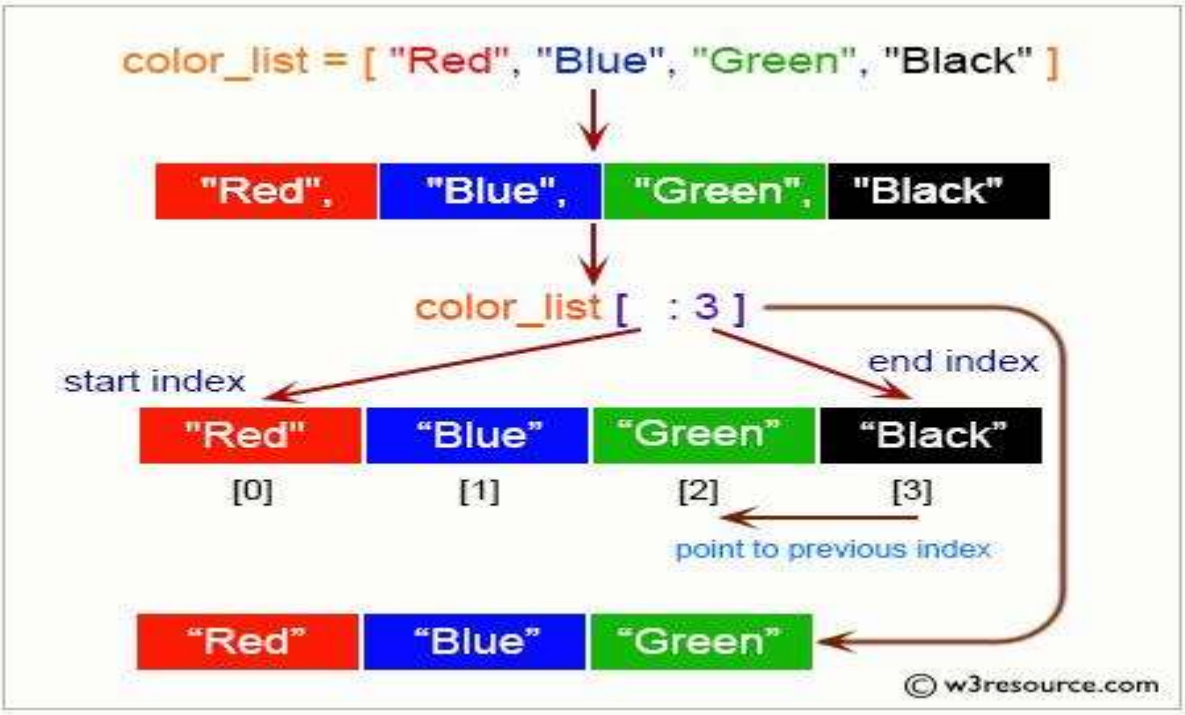
[78, 92, 88]

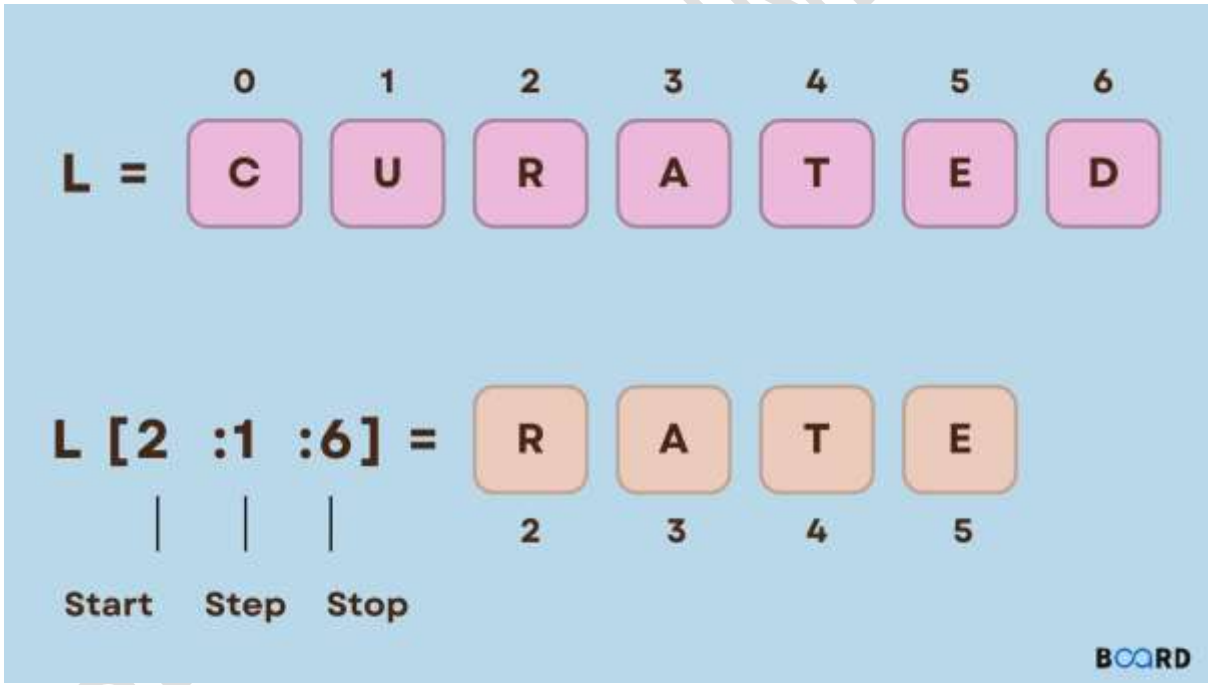
marks[::-1]

Output:

[88, 92, 78, 90, 85]

SUWEBSOFT FOUNDATION


color_list = ["Red", "Blue", "Green", "Black"]
"Red", "Blue", "Green", "Black"
color_list [: 3]
start index
end index
"Red" "Blue" "Green" "Black"
[0] [1] [2] [3]
point to previous index
"Red" "Blue" "Green"
© w3resource.com


0 1 2 3 4 5 6
L = C U R A T E D
L [2 :1 :6] = R A T E
2 3 4 5
Start Step Stop
BOARD

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5

List[0] = 0

List[1] = 1

List[2] = 2

List[3] = 3

List[4] = 4

List[5] = 5

List[0:] = [0,1,2,3,4,5]

List[:] = [0,1,2,3,4,5]

List[2:4] = [2, 3]

List[1:3] = [1, 2]

List[:4] = [0, 1, 2, 3]

7. Adding Elements to a List

Python provides several methods to add data.

A. append()

Adds **one element at the end**.

marks = [80, 85, 90]

marks.append(95)

print(marks)

Output:

[80, 85, 90, 95]

3DWEBSOFT FOUNDATION

B. extend()

Adds multiple elements.

marks = [80, 85]

marks.extend([90, 95, 88])

print(marks)

Output:

[80, 85, 90, 95, 88]

Difference

append([90, 95])

adds the entire list as one element.

Whereas:

extend([90, 95])

adds the elements separately.

C. insert()

Adds an element at a specific position.

marks = [80, 90, 70]

marks.insert(1, 85)

print(marks)

3DWEBSOFT FOUNDATION

Output:

[80, 85, 90, 70]

8. Updating List Elements

Because lists are mutable, we can change values.

marks = [80, 90, 70]

marks[2] = 75

print(marks)

Output:

[80, 90, 75]

9. Removing Elements

remove()

Removes a specific value.

marks = [80, 90, 70, 90]

marks.remove(90)

print(marks)

Output:

[80, 70, 90]

3DWEBSOFT FOUNDATION

It removes the **first matching value**.

pop()

Removes an element using its index.

marks = [80, 90, 70]

marks.pop(1)

print(marks)

Output:

[80, 70]

If no index is specified:

marks.pop()

the last element is removed.

clear()

Removes all elements.

marks = [80, 90, 70]

marks.clear()

print(marks)

Output:

3DWEBSOFT FOUNDATION

[]

10. Useful List Functions

Python provides several useful functions for numerical lists.

Suppose:

marks = [85, 90, 78, 92, 88]

len()

Finds the number of elements.

print(len(marks))

Output:

5

sum()

Finds the total.

print(sum(marks))

Output:

433

max()

Finds the highest value.

print(max(marks))

Output:

92

min()

Finds the lowest value.

3DWEBSOFT FOUNDATION

```
print(min(marks))
```

Output:

78

11. Calculating Average

There is no direct average() function needed for a basic program.

Formula:

Average = Total Marks / Number of Subjects

Python:

```
marks = [85, 90, 78, 92, 88]

total = sum(marks)
average = total / len(marks)

print("Total:", total)
print("Average:", average)
```

Output:

Total: 433

Average: 86.6

12. Searching in a List

Use the in operator.

```
marks = [85, 90, 78, 92, 88]
```

3JWEBSOFT FOUNDATION

print(90 in marks)

Output:

True

Example:

print(100 in marks)

Output:

False

13. Counting Values

The count() method tells us how many times a value appears.

marks = [80, 90, 80, 75, 80]

print(marks.count(80))

Output:

3

14. Finding Position Using index()

marks = [85, 90, 78, 92]

print(marks.index(78))

Output:

2

3DWEBSOFT FOUNDATION

15. Sorting Lists

Ascending Order

```
marks = [85, 70, 95, 80, 90]


marks.sort()


print(marks)
```

Output:

[70, 80, 85, 90, 95]

Descending Order

```
marks.sort(reverse=True)


print(marks)
```

Output:

[95, 90, 85, 80, 70]

sorted()

Another option is:

```
marks = [85, 70, 95, 80]


new_marks = sorted(marks)


print(new_marks)
```

3UWEBSOFT FOUNDATION

The important difference is that sort() changes the original list, while sorted() returns a new sorted list.

16. Reversing a List

```
marks = [85, 90, 78, 92]


marks.reverse()


print(marks)
```

Output:

[92, 78, 90, 85]

17. Copying Lists

Consider:

```
marks = [80, 90, 70]


new_marks = marks
```

Here both variables refer to the same list.

For an independent copy:

```
new_marks = marks.copy()
```

Example:

```
marks = [80, 90, 70]
```

3DWEBSOFT FOUNDATION

```
new_marks = marks.copy()


new_marks[0] = 100


print(marks)
print(new_marks)
```

Output:

```
[80, 90, 70]
[100, 90, 70]
```

18. Nested Lists

A list can contain another list.

Example:

```
marks = [
    [85, 90, 88],
    [75, 80, 82],
    [92, 95, 90]
]
```

This can represent marks of three students.

Student 1 → [85, 90, 88]

Student 2 → [75, 80, 82]

Student 3 → [92, 95, 90]

Access:

3DWEBSOFT FOUNDATION

print(marks[0])

Output:

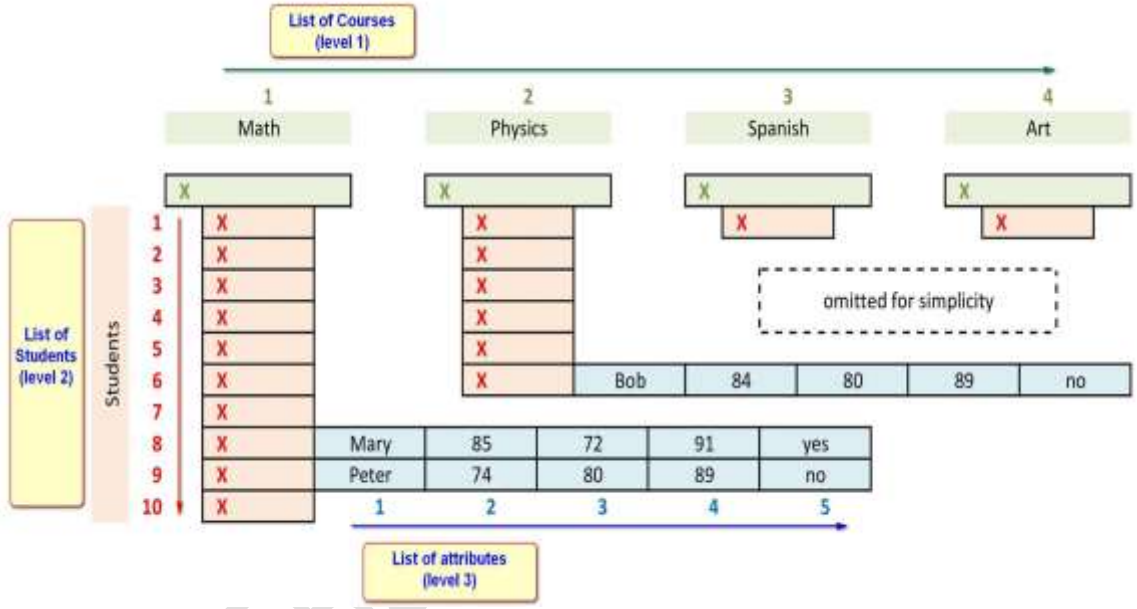
[85, 90, 88]

Access a particular mark:

print(marks[0][1])

Output:

90





Practical Session

Practical 1 – Basic Student Marks Analyzer

Objective

Create a program that stores marks and calculates:

- Total
- Average
- Highest mark
- Lowest mark

Code

```
marks = [85, 90, 78, 92, 88]


total = sum(marks)

average = total / len(marks)

highest = max(marks)

lowest = min(marks)


print("Student Marks:", marks)

print("Total Marks:", total)

print("Average:", average)

print("Highest Mark:", highest)

print("Lowest Mark:", lowest)
```

3DWEBSOFT FOUNDATION

Expected Output

Student Marks: [85, 90, 78, 92, 88]

Total Marks: 433

Average: 86.6

Highest Mark: 92

Lowest Mark: 78

Practical 2 – Enter Marks Using input()

Instead of hard-coding marks, ask the student to enter them.

```
marks = []


for i in range(5):
    mark = int(input("Enter mark: "))
    marks.append(mark)


print("\nMarks:", marks)
print("Total:", sum(marks))
print("Average:", sum(marks) / len(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))
```

Example

Enter mark: 85

Enter mark: 90

9JWEBSOFT FOUNDATION

Enter mark: 78

Enter mark: 92

Enter mark: 88

Marks: [85, 90, 78, 92, 88]

Total: 433

Average: 86.6

Highest: 92

Lowest: 78

Practical 3 – Calculate Grade

Now extend the program to calculate the grade.

```
marks = [85, 90, 78, 92, 88]

average = sum(marks) / len(marks)

if average >= 90:
    grade = "A+"
elif average >= 80:
    grade = "A"
elif average >= 70:
    grade = "B"
elif average >= 60:
```

3DWEBSOFT FOUNDATION

```
    grade = "C"
else:
    grade = "D"

print("Marks:", marks)
print("Average:", average)
print("Grade:", grade)
```

Output:

Marks: [85, 90, 78, 92, 88]

Average: 86.6

Grade: A

Practical 4 – Subject-Wise Marks

We can use a list for subjects and another list for marks.

```
subjects = ["Python", "Maths", "Physics", "English", "Computer"]

marks = [90, 85, 78, 88, 95]

for i in range(len(subjects)):
    print(subjects[i], ":", marks[i])
```

Output:

Python : 90

Maths : 85

3DWEBSOFT FOUNDATION

Physics : 78

English : 88

Computer : 95

Practical 5 – Find Pass and Fail Subjects

Assume the pass mark is 40.

```
subjects = ["Python", "Maths", "Physics", "English", "Computer"]
marks = [90, 35, 78, 88, 95]

for i in range(len(marks)):
    if marks[i] >= 40:
        print(subjects[i], "-", marks[i], "- PASS")
    else:
        print(subjects[i], "-", marks[i], "- FAIL")
```

Output:

Python - 90 - PASS

Maths - 35 - FAIL

Physics - 78 - PASS

English - 88 - PASS

Computer - 95 - PASS

Practical 6 – Find the Top 3 Marks

3DWEBSOFT FOUNDATION

```
marks = [75, 92, 85, 98, 67, 88, 95]

marks.sort(reverse=True)

print("Top 3 Marks:", marks[:3])
```

Output:

Top 3 Marks: [98, 95, 92]

Advanced Challenge – Student Marks Analyzer

Ask students to create a complete application.

Requirements

The program should:

1. Ask for student name.
2. Ask for marks in 5 subjects.
3. Store marks in a list.
4. Calculate total.
5. Calculate average.
6. Find highest mark.
7. Find lowest mark.
8. Display pass/fail.
9. Calculate grade.
10. Display all results neatly.

Sample Program

3DWEBSOFT FOUNDATION

```
student_name = input("Enter student name: ")

marks = []

for i in range(5):
    mark = int(input(f"Enter mark {i + 1}: "))
    marks.append(mark)

total = sum(marks)
average = total / len(marks)
highest = max(marks)
lowest = min(marks)

if average >= 90:
    grade = "A+"
elif average >= 80:
    grade = "A"
elif average >= 70:
    grade = "B"
elif average >= 60:
    grade = "C"
else:
    grade = "D"
```

3UWEBSOFT FOUNDATION

```
print("\n===== STUDENT REPORT =====")
print("Name:", student_name)
print("Marks:", marks)
print("Total:", total)
print("Average:", average)
print("Highest:", highest)
print("Lowest:", lowest)
print("Grade:", grade)
```

Interactive Activity – "List Detective"

Give students this list:

```
marks = [78, 92, 85, 92, 67, 88, 78]
```

Ask them to predict the answers **before running the program**.

Question 1

```
print(marks[2])
```

Question 2

```
print(marks[-1])
```

Question 3

```
print(len(marks))
```

Question 4

```
print(max(marks))
```

3DWEBSOFT FOUNDATION

Question 5

print(min(marks))

Question 6

print(marks.count(92))

Question 7

print(marks[1:5])

This activity helps students understand indexing and slicing instead of simply memorizing syntax.

Classroom Challenge – Improve the Analyzer

Ask students to add these features:

Level 1

Display total and average.

Level 2

Display highest and lowest marks.

Level 3

Display grade.

Level 4

Display PASS/FAIL.

Level 5

Sort marks from highest to lowest.

Level 6

Display the top 3 marks.

3DWEBSOFT FOUNDATION

Level 7

Allow the user to enter any number of subjects.

Important List Methods

Method	Purpose	Example
append()	Add one item	marks.append(90)
extend()	Add multiple items	marks.extend([80,90])
insert()	Add at position	marks.insert(1,85)
remove()	Remove value	marks.remove(85)
pop()	Remove by index	marks.pop(2)
clear()	Remove all	marks.clear()
index()	Find position	marks.index(90)
count()	Count occurrences	marks.count(90)
sort()	Sort list	marks.sort()
reverse()	Reverse list	marks.reverse()
copy()	Copy list	marks.copy()

3DWEBSOFT FOUNDATION

Quick Quiz

1. What is a list?

Answer: A list is an ordered, mutable collection used to store multiple values.

2. What is the first index of a Python list?

Answer: 0

3. Which method adds one item to the end?

Answer: append()

4. Which method removes the last element?

Answer: pop()

5. Which function finds the highest value?

Answer: max()

6. Which function finds the total?

Answer: sum()

7. Are lists mutable?

Answer: Yes.

8. What does len(marks) return?

Answer: Number of elements in the list.

9. What does marks[::-1] do?

Answer: Returns the list in reverse order.

10. What is a nested list?

Answer: A list containing one or more lists.

3DWEBSOFT FOUNDATION

Homework

Task 1 – Marks Calculator

Create a program that accepts **6 subject marks** and displays:

- All marks

- Total
- Average
- Highest
- Lowest
- Grade

Task 2 – Shopping List

Create:

shopping = []

Allow the user to:

- Add items
- Remove items
- Display all items
- Count items

Task 3 – Topper Finder

Given:

marks = [78, 95, 88, 92, 67, 99, 85]

Write a program to find:

- Highest mark
- Lowest mark
- Top 3 marks
- Average

2-Hour Class Plan

3DWEBSOFT FOUNDATION

Time	Activity
0–10 min	Previous class revision
10–25 min	Introduction to Lists
25–40 min	Indexing & Slicing
40–55 min	List methods
55–70 min	Functions: sum(), max(), min(), len()
70–90 min	Student Marks Analyzer
90–105 min	Advanced challenges
105–115 min	Quiz & List Detective
115–120 min	Homework & recap

Real-World Applications of Lists

Lists are extremely important in programming because they allow us to work with collections of data.

Examples:

Student marks

↓

[85, 90, 78, 92, 88]

Product prices

↓

[499, 799, 1299, 599]

3DWEBSOFT FOUNDATION

Employee salaries

↓

[25000, 32000, 28000, 45000]

AI training data

↓

[data1, data2, data3, data4]

Sensor readings

↓

[24.5, 25.1, 24.8, 26.2]

Lists are therefore an important foundation for **Data Science, Machine Learning and AI**, where programs frequently work with collections of data.

Key Takeaways

Students should remember:

List

↓

Ordered

↓

Mutable

3DVEBSOFT FOUNDATION

↓

Allows duplicates

↓

Supports indexing

↓

Supports slicing

↓

Can add/remove/update data

↓

Can sort and search

↓

Can analyze data

↓

Can be used in real-world applications

Most Important Syntax

```
marks = [85, 90, 78, 92]


marks.append(95)
marks.insert(1, 88)
marks.remove(78)


print(marks[0])
print(marks[-1])
```

3DWEBSOFT FOUNDATION

```
print(marks[1:3])


print(len(marks))
print(sum(marks))
print(max(marks))
print(min(marks))


marks.sort()
marks.reverse()
```

End-of-Class Project

Build a complete Student Marks Analyzer using Python Lists.

Input → Store in List → Process → Analyze → Display Report

3DWEBSOFT FOUNDATION